Practice 07 — Launch a Web Server on EC2

Objective: launch a server that sets itself up on first boot, reach it over HTTP and SSH, see what changes when you stop and start it, and then remove everything. These are the skills from today's lecture.

Timebox: ~45–60 min, in class. If you're stuck on one step for more than 10 minutes, ask instead of pushing through alone.

> **How you're checked:** this practice is done in class and **checked orally** at the end. There's no written report. Keep your terminal (and console tabs) open, be ready to **show** each item in the Definition of done and **explain** what each step does. The instructor will ask you a few of the questions under "Be ready to explain".

> **AWS access isn't guaranteed for everyone, so there are two tracks.** Do **Track A** if you have an AWS account (the one you secured last week). Otherwise do **Track B**, which runs a VM on your own laptop. Tell the instructor which track you did.

Definition of done (things you can show)

- ☐ Your user data / cloud-init file, and you can explain each line
- ☐ The web page served by **your** server, showing your name (Track A: also the instance ID and AZ)
- ☐ A shell on the server, and the end of the cloud-init log
- ☐ The address **before and after** a stop → start, and what changed
- ☐ Cleanup: nothing of yours is still running (Track A: no instance, security group or key pair left)

Track A — real AWS

Use region `eu-central-1` **(Frankfurt)** and last week's CLI profile.

```
export AWS_PROFILE=css4003
export AWS_REGION=eu-central-1
aws sts get-caller-identity          # must be your IAM admin user, not root
```

A1. Security group: SSH from you, HTTP from anyone

```
SG=$(aws ec2 create-security-group --group-name web-sg \
  --description "SSH from me, HTTP from anyone" \
  --query GroupId --output text)
MYIP=$(curl -s https://checkip.amazonaws.com)
echo "$SG  $MYIP"
aws ec2 authorize-security-group-ingress --group-id $SG --protocol tcp --port 22 --cidr $MYIP/32
aws ec2 authorize-security-group-ingress --group-id $SG --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 describe-security-groups --group-ids $SG \
  --query 'SecurityGroups[0].IpPermissions[].[FromPort,IpRanges[0].CidrIp]' --output table
```

A2. Key pair

```
aws ec2 create-key-pair --key-name css4003-key \
  --query KeyMaterial --output text > css4003-key.pem
chmod 400 css4003-key.pem
```

Windows PowerShell: `ssh` refuses a key that other users can read. If you get "UNPROTECTED PRIVATE KEY FILE", run `icacls css4003-key.pem /inheritance:r /grant:r "$($env:USERNAME):R"`.

A3. User data

Save as `user-data.sh`. Put **your name** in the `NAME=` line:

```
#!/bin/bash
NAME="Your Name"
dnf install -y nginx
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/availability-zone)
echo "<h1>CSS4003 — $NAME</h1><p>Instance $IID in $AZ ($(hostname -f))</p>" \
  > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

The script asks the **instance metadata service** (IMDSv2: token first, then the request) who it is, and writes that into the page.

A4. Launch

```
IID=$(aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --instance-type t3.micro \
  --key-name css4003-key \
  --security-group-ids $SG \
  --user-data file://user-data.sh \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=css4003-web}]' \
  --query 'Instances[0].InstanceId' --output text)
echo $IID
aws ec2 wait instance-running --instance-ids $IID
aws ec2 describe-instances --instance-ids $IID \
  --query 'Reservations[0].Instances[0].
[State.Name,PublicIpAddress,PrivateIpAddress,Placement.AvailabilityZone]' \
  --output text
```

Check the free-tier-eligible instance types for **your** account on https://aws.amazon.com/free first. If `t3.micro` isn't one of them, use one that is.

A5. Use it

```
IP=<public-ip from A4>
curl http://$IP                # wait 1–2 min: "running" comes before cloud-init finishes
```

Open `http://<public-ip>` in a browser too. Then SSH in:

```
ssh -i css4003-key.pem ec2-user@$IP
# on the instance:
cloud-init status                             # want: status: done
sudo tail -n 20 /var/log/cloud-init-output.log
systemctl is-active nginx
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/public-ipv4
exit
```

A6. Stop → start: what changes?

```
aws ec2 stop-instances --instance-ids $IID
aws ec2 wait instance-stopped --instance-ids $IID
aws ec2 start-instances --instance-ids $IID
aws ec2 wait instance-running --instance-ids $IID
aws ec2 describe-instances --instance-ids $IID \
  --query 'Reservations[0].Instances[0].[PublicIpAddress,PrivateIpAddress]' --output text
```

Compare with A4. Then `curl` the **new** public IP. The page still works without re-running user data. Why?

A7. Cleanup

```
aws ec2 terminate-instances --instance-ids $IID
aws ec2 wait instance-terminated --instance-ids $IID
aws ec2 delete-security-group --group-id $SG      # only works after the instance is gone
aws ec2 delete-key-pair --key-name css4003-key
rm -f css4003-key.pem

# prove nothing is left
aws ec2 describe-instances \
  --filters Name=instance-state-name,Values=pending,running,stopping,stopped \
  --query 'Reservations[].Instances[].InstanceId' --output text     # empty
aws ec2 describe-addresses --query 'Addresses[].PublicIp' --output text   # empty
```

Stretch (optional), before cleanup: turn on termination protection with `aws ec2 modify-instance-attribute --instance-id $IID --disable-api-termination`, try to terminate, read the error, then turn it off with `--no-disable-api-termination`.

Track B — no AWS (Multipass VM)

Multipass runs Ubuntu VMs on your laptop and uses **cloud-init**, the same tool that runs EC2 user data.

EC2 (Track A)	Multipass (Track B)
AMI	Ubuntu image (default: latest LTS)
`run-instances --user-data`	`multipass launch --cloud-init`
Public IP	VM's IP, reachable only from your laptop
`ssh -i key.pem ec2-user@...`	`multipass shell web`
Security group	None by default: only your machine can reach the VM
Instance metadata (IMDSv2)	No equivalent
stop / start	`multipass stop` / `multipass start`
terminate	`multipass delete` + `multipass purge`

B1. Install

- macOS: `brew install --cask multipass` (or the installer from the site)
- Windows: the installer from the site (uses Hyper-V, or VirtualBox on Home editions)
- Linux: `sudo snap install multipass`

```
multipass version
```

A VM needs about 1 GB RAM and a few GB of disk.

B2. cloud-init file

Save as `web.yaml`. Put **your name** in the page:

```
#cloud-config
packages:
  - nginx
runcmd:
  - echo "<h1>CSS4003 — Your Name</h1><p>Host $(hostname -f), launched by Multipass</p>" >
/var/www/html/index.html
  - systemctl enable --now nginx
```

On Ubuntu the web root is `/var/www/html`, not `/usr/share/nginx/html` as on Amazon Linux.

B3. Launch and use it

```
multipass launch --name web --cloud-init web.yaml
multipass list                    # state + IPv4
multipass info web                # more detail, including the IPv4
curl http://<vm-ip>
```

Then open a shell:

```
multipass shell web
# inside the VM:
cloud-init status                              # want: status: done
sudo tail -n 20 /var/log/cloud-init-output.log
systemctl is-active nginx
exit
```

Or run one command without a shell: `multipass exec web -- systemctl is-active nginx`.

B4. Stop → start

```
multipass info web | grep IPv4     # note it
multipass stop web
multipass list                     # Stopped
multipass start web
multipass info web | grep IPv4     # compare
curl http://<vm-ip>
```

On your laptop the IP often stays the same. On EC2 the auto-assigned public IP **changes** on every stop → start. Be ready to say why that difference matters for a real website.

B5. Cleanup

```
multipass delete web
multipass purge                    # frees the disk; like "terminated"
multipass list                     # no "web"
```

Last resort: Docker (only if Multipass won't install)

This is **weaker**: no VM, no cloud-init, no boot, no lifecycle to compare. You only practice "a server that serves your page".

```
mkdir -p site
echo "<h1>CSS4003 — Your Name</h1>" > site/index.html
docker run -d --name web -p 8080:80 -v "$PWD/site:/usr/share/nginx/html:ro" nginx:alpine
curl http://localhost:8080
docker stop web && docker start web && curl http://localhost:8080
docker rm -f web
```

Tell the instructor you did this fallback, and be ready to explain what cloud-init would have done that this doesn't.

Be ready to explain

1. What does user data do, when does it run, and where do you look when it "didn't work"?
2. Why is port 22 open only to your IP, while port 80 is open to `0.0.0.0/0`?
3. SSH says `Permission denied (publickey)`. And if it says `Connection timed out`? What's the likely cause of each?
4. What changed after stop → start, and what stayed the same? Why does the page still work without user data running again?
5. Stop vs terminate: what happens to the root disk, and what still costs money while an instance is stopped?
6. Why does the user data fetch a **token** before asking `169.254.169.254` anything?

Common mistakes

- **Page doesn't load right after launch.** "Running" comes before cloud-init finishes installing nginx. Wait a minute or two, then check `cloud-init status`.
- **`curl https://...` fails.** There's no TLS on this server. Use `http://`.
- **SSH times out.** Your public IP changed (new Wi-Fi, phone hotspot), so the `/32` rule no longer matches. Re-run the `checkip` command and add a rule for the new address. Also check you're using the **current** public IP after a stop → start.
- **`Permission denied (publickey)`.** Wrong user (`ec2-user` on Amazon Linux, `ubuntu` on Ubuntu), wrong key, or the `.pem` is readable by others (`chmod 400`).
- **Edited `user-data.sh` and nothing changed.** User data only runs on the first boot. Terminate and launch a new instance.
- **`delete-security-group` fails with `DependencyViolation`.** The instance isn't fully terminated yet. Wait for `instance-terminated` first.
- **The `.pem` ends up in Git.** Never commit it. Add `*.pem` to `.gitignore`.
- **Track B: `launch failed: ... timed out`.** The VM is slow to boot on first run while it downloads the image. Run `multipass list`. It often finished anyway. Otherwise `multipass delete --purge web` and try again.